

OperisChain

Especificación Técnica · Prototipo v1.0

Autor	Diego Oquendo
Versión	1.0 — Febrero 2025
Objetivo	Guía completa de arquitectura, stack y hoja de ruta para construir el prototipo con Claude Code
Vertical	Freight Forwarders & Agencias de Carga — Colombia + LatAm

Este documento contiene todo lo necesario para iniciar la construcción del prototipo de OperisChain: la idea de negocio, la arquitectura del sistema, el stack tecnológico completo en JavaScript/Node.js, la estructura de carpetas, los primeros pasos con Claude Code, y la hoja de ruta semana a semana.

La Oportunidad

Las empresas de carga y logística en Colombia y LatAm enfrentan un problema universal: operaciones altamente documentales manejadas con procesos manuales del siglo pasado. Cotizaciones armadas desde correos dispersos, contratos perdidos en carpetas, códigos HS buscados a mano, y equipos de operadores expertos que pasan el 60% de su día haciendo trabajo repetitivo.

Las soluciones enterprise existentes (Raft, Reform, Augment) apuntan al mercado de EE.UU. y Europa con contratos anuales y procesos de onboarding de 6 meses. El mercado mid-market latinoamericano está completamente desatendido.

Propuesta de Valor

OperisChain reduce el overhead operativo de agencias de carga en 30% automatizando los flujos documentales más costosos en tiempo: generación de cotizaciones, búsqueda semántica sobre contratos y SOPs, triaje de correos, y validación de cumplimiento aduanero.

Segmento Objetivo

- Freight forwarders medianos — 5 a 50 empleados
- Agencias de aduana con alto volumen documental
- Brokers de carga con operaciones Colombia — mundo
- Importadores/exportadores con equipos ops propios

Modelo de Negocio

- **Auditoría Operativa: \$2,500 – \$3,500 USD (puerta de entrada)**
 - Mapeo de flujos, identificación de cuellos de botella, prototipo en vivo
- **Implementación AI completa: \$12,000 – \$22,000 USD**
 - Sistema funcionando con datos reales del cliente en 3 semanas
- **Evolución Continua: \$1,500 – \$2,500 USD/mes**
 - Mantenimiento, optimización, nuevos flujos automatizados

Ventaja Competitiva

- Mercado desatendido: ningún competidor apunta a mid-market LatAm
- Implementación en semanas, no meses
- Sistema entrenado con los datos reales del cliente, no genérico
- Construido por un solo desarrollador full-stack con Claude Code — velocidad máxima

Arquitectura de Producción

El sistema sigue una arquitectura de 6 capas: ingesta de documentos, procesamiento y embedding, almacenamiento vectorial, orquestación con LLM, automatización de flujos, y capa de presentación. Todo en Node.js con TypeScript para máxima velocidad de desarrollo.

Capa 1 — Ingesta de Documentos

Punto de entrada para todos los documentos del cliente. Soporta múltiples formatos y fuentes.

- PDFs: facturas comerciales, BL, guías aéreas, certificados de origen
- Excel / CSV: tarifas de carriers, listas de precios, bases de datos de HS codes
- Correos: Gmail API o IMAP para parseo de emails con tarifas y solicitudes
- Word / texto plano: SOPs, contratos, instructivos operativos
- Formularios web: portal de cliente para ingreso directo

Capa 2 — Procesamiento y Embedding

Pipeline de transformación: el documento crudo se convierte en chunks semánticos con metadata.

- Extracción de texto con LangChain.js document loaders
- Chunking recursivo (1000 tokens, overlap 200) para documentos largos
- Chunking semántico para contratos y tarifas estructuradas
- Embedding con text-embedding-3-small de OpenAI (más barato, suficiente para este caso)
- Metadata enriquecida: tipo de doc, fecha, cliente, carrier, ruta

Capa 3 — Almacenamiento

Tres tipos de almacenamiento con responsabilidades distintas:

- **Vector DB (Pinecone): embeddings para búsqueda semántica**
 - Índices separados por cliente para aislamiento de datos
 - Namespace por tipo de documento (contratos, tarifas, SOPs)
- **PostgreSQL (Supabase): metadata estructurada, historial, usuarios**
 - Tablas: documentos, conversaciones, cotizaciones, clientes
- **Redis (Upstash): caché de respuestas frecuentes, rate limiting**
 - TTL de 24h para búsquedas repetidas en el mismo día
- **S3 / Cloudflare R2: archivos originales en raw storage**

Capa 4 — Orquestación con LLM

El cerebro del sistema. Recibe queries, recupera contexto relevante, genera respuestas estructuradas.

- LangChain.js LCEL para construcción de chains y agentes
- Router de modelos: GPT-4o para generación, Claude claude-sonnet-4-6 para análisis documental complejo
- RetrievalQA con citación de fuentes obligatoria
- Structured output con Zod schemas (TypeScript-native)
- Guardrails: validación de outputs antes de entrega al usuario
- LangSmith para trazabilidad completa de cada llamada

Capa 5 — Automatización de Flujos

Los cuatro módulos que se construyen en el prototipo:

- **Quote Generator: emails con tarifas → cotización estructurada en JSON → PDF**
 - Input: 5-10 correos con tarifas de carriers
 - Output: cotización con columnas carrier, ruta, precio, tiempo tránsito
- **Document Q&A: pregunta en lenguaje natural → respuesta con fuente citada**
 - Ejemplo: '¿Cuál es el tiempo de tránsito de Aerolínea X a Miami?'
- **Email Triage: clasificación automática → prioridad → routing**
 - Categorías: cotización urgente, consulta, confirmación, seguimiento
- **Compliance Assistant: descripción de producto → HS code + validación DIAN**
 - Integración con VUCE y normativa colombiana de comercio exterior

Capa 6 — Interfaces

- Dashboard web: Next.js 14 con App Router
- Chat interface: componente de conversación sobre documentos
- API REST: endpoints para integración con TMS externos
- Webhooks: notificaciones a Slack / WhatsApp Business
- Monitoring dashboard: uso, costos, savings calculados en tiempo real

Stack — JavaScript / Node.js

Todo el sistema se construye en TypeScript con Node.js. Esta decisión aprovecha tu fortaleza principal y maximiza la velocidad con Claude Code.

Backend & API

Capa	Tecnología	Paquete / Versión	Propósito
Backend	Node.js + TypeScript	node 20 LTS + tsx	Runtime principal
Framework API	Hono.js	hono@latest	Ultra-fast, edge-ready, mejor que Express para este caso
Validación	Zod	zod@3	Schema validation + structured LLM outputs
Auth	Clerk	@clerk/backend	Auth completo sin escribir una línea de auth
ORM	Drizzle ORM	drizzle-orm@latest	TypeScript-native, más rápido que Prisma

IA & LLM

Capa	Tecnología	Paquete / Versión	Propósito
LLM Primario	OpenAI GPT-4o	openai@4	Cotizaciones, triaje, Q&A general
LLM Secundario	Anthropic Claude	claude-sonnet-4-6	Análisis documental complejo, compliance
Orquestación	LangChain.js	langchain@0.2	Chains, agentes, memory management
Embeddings	text-embedding-3-small	openai@4	\$0.02/1M tokens — óptimo para este volumen
Trazabilidad	LangSmith	langsmith@latest	Logging, evaluación, debug de chains
Structured Output	Zod + LangChain	zod-to-json-schema	Outputs tipados y validados

Base de Datos & Storage

Capa	Tecnología	Paquete / Versión	Propósito
Vector DB	Pinecone	pinecone-client@3	Búsqueda semántica, índices por cliente
SQL	PostgreSQL vía Supabase	drizzle-orm/pg-core	Metadata, historial, usuarios
Caché	Redis vía Upstash	@upstash/redis	Serverless Redis, caché de queries

File Storage	Cloudflare R2	aws-sdk/client-s3	Compatible S3, más barato que AWS
--------------	---------------	-------------------	-----------------------------------

Procesamiento de Documentos

Capa	Tecnología	Paquete / Versión	Propósito
PDF Parsing	pdf-parse + pdfjs-dist	pdf-parse@1.1	Extracción de texto de PDFs
Excel/CSV	xlsx (SheetJS)	xlsx@latest	Parseo de tarifas en Excel
Email parsing	mailparser	mailparser@3	Parseo de correos MIME
IMAP/Gmail	imapflow + googleapis	imapflow@1	Conexión a bandeja del cliente
OCR	tesseract.js	tesseract.js@5	PDFs escaneados sin texto
Doc chunking	langchain/text_splitter	langchain@0.2	Chunking recursivo y semántico

Frontend

Capa	Tecnología	Paquete / Versión	Propósito
Framework	Next.js 14	next@14	App Router, SSR, API routes
UI Components	shadcn/ui	shadcn-ui@latest	Componentes listos, totalmente customizable
Estilos	Tailwind CSS	tailwindcss@3	Utility-first, rápido de construir
Chat UI	Vercel AI SDK	ai@3	Streaming responses, chat components
Tablas/Datos	TanStack Table	@tanstack/react-table@8	Tablas de cotizaciones y documentos
Forms	React Hook Form + Zod	react-hook-form@7	Forms tipados y validados

Infraestructura & DevOps

Capa	Tecnología	Paquete / Versión	Propósito
Deploy Backend	Railway.app	—	Deploy en 1 click, PostgreSQL incluido
Deploy Frontend	Vercel	—	Next.js nativo, edge functions
CI/CD	GitHub Actions	—	Lint, test, deploy automático
Monitoring	Better Stack	—	Logs, uptime, alertas
Costos LLM	LangSmith + OpenAI dashboard	—	Control de costos por cliente

Herramientas de Desarrollo

Capa	Tecnología	Paquete / Versión	Propósito
IDE + AI	Claude Code	—	Agente de código principal — tu decisión correcta
Linting	ESLint + Prettier	eslint@8	Código consistente
Testing	Vitest	vitest@1	Tests unitarios ultrarrápidos
API Testing	Bruno	—	Alternativa open-source a Postman
Env vars	dotenv + t3-env	@t3-oss/env-nextjs	Variables de entorno tipadas

El Primer Módulo a Construir

El Quote Generator es la pieza de mayor impacto para la demo de auditoría. Un freight forwarder que ve cómo su sistema genera una cotización desde 8 correos en 30 segundos cierra el contrato en esa misma reunión.

Flujo del Quote Generator

1. El usuario sube 5-10 correos con tarifas de carriers (texto o forward)
2. El pipeline parsea cada correo con mailparser
3. Los chunks de tarifas se embebean y almacenan en Pinecone
4. El usuario define: origen, destino, tipo de carga, peso/volumen
5. El chain de LangChain recupera las tarifas relevantes por ruta
6. GPT-4o genera una cotización estructurada en JSON con Zod schema
7. El frontend renderiza la tabla de cotización comparativa
8. El usuario puede exportar a PDF o copiar al clipboard

Schema de Cotización (Zod)

```
const QuoteLineSchema = z.object({
  carrier: z.string(),
  route: z.string(),
  transitDays: z.number(),
  priceUSD: z.number(),
  validUntil: z.string(),
  notes: z.string().optional(),
  sourceDocument: z.string(), // cita del documento origen
});

const QuoteSchema = z.object({
  quoteId: z.string(),
  generatedAt: z.string(),
  cargo: z.object({ origin: z.string(), destination: z.string() }),
  lines: z.array(QuoteLineSchema),
  recommended: z.string(), // carrier recomendado
  reasoning: z.string(),    // por qué se recomienda
});
```

Prompt del Chain (Estructura)

El prompt del Quote Generator tiene tres secciones críticas:

- System prompt: rol de especialista en logística internacional, instrucción de citar fuentes siempre
- Context injection: chunks recuperados de Pinecone con las tarifas relevantes

- User instruction: parámetros de la cotización + instrucción de output en JSON con el schema Zod

Guía de Trabajo con Claude Code

Claude Code es tu decisión correcta para este proyecto. Es el entorno ideal para construir sistemas con LangChain porque el agente puede leer la documentación, escribir el código, ejecutarlo, ver el error, y corregirlo en un solo loop sin salir del terminal.

Por qué Claude Code para este proyecto

- Manejo nativo de TypeScript y Node.js — entiende el stack completo
- Puede leer y aplicar la documentación de LangChain.js directamente
- Loop de corrección autónomo: escribe, ejecuta, ve el error, corrige
- Ideal para construir chains de LLM donde el debugging es iterativo
- Maneja el contexto del proyecto completo, no solo un archivo

Setup Inicial con Claude Code

Cuando abras Claude Code en tu proyecto, dale este contexto inicial:

```
Eres el arquitecto técnico de OperisChain, un sistema de automatización
de flujos operativos para freight forwarders en Colombia.
Stack: Node.js 20 + TypeScript, Hono.js, LangChain.js, OpenAI, Pinecone,
Drizzle ORM con PostgreSQL (Supabase), Next.js 14.
Siempre usa TypeScript estricto, Zod para schemas, y cita fuentes en
todos los outputs del LLM.
```

Comandos de Inicio Recomendados

- Primero pídele que genere la estructura de carpetas completa
- Luego el schema de Drizzle para las tablas base (documents, quotes, clients)
- Luego el pipeline de ingesta de PDFs con LangChain document loaders
- Luego el chain del Quote Generator con Zod structured output
- Por último el frontend con Next.js + Vercel AI SDK para el chat

Prompts Clave para Claude Code

Quote Generator chain:

```
Construye el quote.chain.ts en LangChain.js LCEL que:
1. Recibe un array de strings (contenido de correos con tarifas)
2. Hace embed de cada correo y los guarda en Pinecone namespace 'rates'
3. Dado un objeto {origin, destination, cargoType, weight}, recupera
   las tarifas relevantes con RetrievalQA
4. Genera output estructurado con el QuoteSchema de Zod
5. Cada línea debe incluir sourceDocument citando el correo origen
```

Document ingestion pipeline:

Construye el pipeline de ingesta en `src/ingestion/index.ts` que:

1. Acepta un archivo (PDF, Excel, Word, texto plano)
2. Detecta el tipo automáticamente
3. Extrae el texto con el loader apropiado de LangChain
4. Hace chunking recursivo (1000 tokens, overlap 200)
5. Genera embeddings con `text-embedding-3-small`
6. Guarda en Pinecone con metadata: {clientId, docType, fileName, date}
7. Guarda el record en PostgreSQL via Drizzle

Configuración de Variables

Todas las claves necesarias para el prototipo. Consíguelas en este orden: primero las gratuitas, luego las de pago.

```
# LLM APIs
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
LANGSMITH_API_KEY=ls-...
LANGSMITH_PROJECT=operischain-dev

# Vector DB
PINECONE_API_KEY=...
PINECONE_INDEX=operischain
PINECONE_ENVIRONMENT=...

# Database
DATABASE_URL=postgresql://... # Supabase connection string

# Cache
UPSTASH_REDIS_REST_URL=...
UPSTASH_REDIS_REST_TOKEN=...

# File Storage
R2_ACCOUNT_ID=...
R2_ACCESS_KEY_ID=...
R2_SECRET_ACCESS_KEY=...
R2_BUCKET_NAME=operischain-docs

# Auth
CLERK_SECRET_KEY=sk_...
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=pk_...

# App
NODE_ENV=development
API_URL=http://localhost:3001
```

Para el prototipo de auditoría puedes simplificar: solo necesitas OPENAI_API_KEY, ANTHROPIC_API_KEY, y PINECONE. Agrega las demás en la implementación completa.

Plan de Construcción — 6 Semanas

Esta es la hoja de ruta para tener el prototipo listo para la primera auditoría de cliente. El objetivo de la semana 2 es tener algo que puedas mostrar en una reunión.

Semana	Fase	Entregables	Resultado
Sem 1	Setup & Ingesta	Monorepo, env vars, pipeline de ingesta PDF/Excel, Pinecone index	Cualquier PDF se ingesta y es queryable
Sem 2	Quote Generator	LangChain chain, Zod schema, UI básica en Next.js, demo funcional	Demo lista para mostrar a cliente
Sem 3	Document Q&A	Chat interface con Vercel AI SDK, citación de fuentes, historial	Chat sobre documentos del cliente
Sem 4	Email Triage	IMAP connection, clasificación automática, routing por categoría	Emails clasificados automáticamente
Sem 5	Compliance	HS code lookup, validación DIAN, output estructurado	Asistente de cumplimiento aduanero
Sem 6	Polish & Deploy	Dashboard de métricas, deploy Railway + Vercel, monitoring	Sistema en producción para cliente 1

Meta de la Semana 2

Al final de la semana 2 debes tener algo que puedas mostrar en WhatsApp en un video de 60 segundos: cargar 5 correos con tarifas, pedir una cotización, ver la tabla generada. Ese video es tu herramienta de ventas más poderosa.

Estimado de Costos Mensuales en Producción

- OpenAI API (GPT-4o + embeddings): \$30 – \$80 USD/mes por cliente activo
- Pinecone Starter: \$0 (hasta 1 índice) → \$70/mes en escala
- Supabase Pro: \$25/mes
- Railway (backend): \$5 – \$20/mes
- Vercel Pro: \$20/mes
- Upstash Redis: \$0 – \$10/mes
- LangSmith Developer: \$0 (hasta 5k traces)

Total estimado: \$80 – \$150 USD/mes por los primeros 2-3 clientes. Muy por debajo del pricing que cobras.

Empieza Aquí

Abre Claude Code, posícionalte en una carpeta vacía, y ejecuta este primer prompt:

```
Crea un monorepo con Turborepo para el proyecto OperisChain con esta estructura:
```

- apps/api: Hono.js + TypeScript backend
- apps/web: Next.js 14 App Router frontend
- packages/shared: tipos TypeScript compartidos

```
Configura:
```

- TypeScript estricto en todo el proyecto
- ESLint + Prettier
- dotenv para variables de entorno
- Script 'dev' que levanta ambos apps en paralelo

```
En el API, instala desde el inicio: hono, langchain, @langchain/openai,  
@langchain/pinecone, openai, @pinecone-database/pinecone,  
drizzle-orm, pdf-parse, xlsx, mailparser, zod
```

Con ese solo prompt, Claude Code generará toda la estructura base del proyecto. Desde ahí construyes módulo por módulo siguiendo el orden de la hoja de ruta.